becomes difficult and expensive.

Difficult, because it becomes like solving a puzzle: every time we rotate an internal node, we potentially cause lower levels in the subtrees pushed down to become more unbalanced, so it’s not trivial to come up with the right order of rotations to obtain the best possible tree structure.

Expensive, because we need to keep track of the height of each subtree, and because coming up with a sequence of rotations requires extra work.

In the previous section, we used the term “*tendentially-balanced*” to describe the result we can get; this has probably already revealed the key point to the eyes of the most observant readers: we are talking about introducing a randomized element in our data structures.

Randomness will be a constant factor in this first part of the book, we’ll see several data structures leveraging it, including Bloom filters and skip lists; to help all readers be comfortable with the topic, we prepared a short introduction to randomized algorithms in appendix F: feel free to take a look before delving into this section.